Project Documentation

File: CannyIEC.library

Date: 12/10/2026

Profile: CODESYS V3.5 SP22 Patch 3

Table of contents

1 Folder: CannyIEC 3
1.1 Folder: Enums 3
1.1.1 DUT: ERROR 3
1.2 Folder: Function Blocks 3
1.2.1 POU: AsymmetricDebounce 3
1.2.2 POU: DIF 4
1.2.3 POU: FirstCycle 4
1.2.4 POU: FLIP_FLOP 4
1.2.5 POU: PT1 5
1.2.6 POU: SymmetricDebounce 5
1.2.7 POU: WAS 6
1.3 Folder: Functions 7
1.3.1 POU: ISIGNBIT 7
1.3.2 POU: REPSILONEQ 7
1.3.3 POU: RSIGNBIT 8
1.4 Folder: GlobalConstants 8
1.4.1 Exchange Variable List: Constants 8
1.5 Folder: GlobalVariables 8
1.5.1 Exchange Variable List: Variables 9
1.6 Folder: Interfaces 9
1.7 Folder: Structs 9
1.8 Folder: Types 9
2 Folder: Library Information 9
2.1 POU: GetLibVersion 9
2.2 POU: GetLibVersionNumber 9
2.3 POU: IsLibReleased 10
3 Library Manager: Library Manager 10
4 Project Information: Project Information 10
5 : Project Settings 11

1 Folder: CannyIEC

1.1 Folder: Enums

1.1.1 DUT: ERROR

```
{attribute 'qualified_only'}
{attribute 'conditionalshow'}
{attribute 'conditionalshow' := 'DevMode'}
{attribute 'hide'}
TYPE ERROR : (
        NO_ERROR,
        TIME_OUT
    );
END_TYPE

```

1.2 Folder: Function Blocks

1.2.1 POU: AsymmetricDebounce

```
FUNCTION_BLOCK AsymmetricDebounce
VAR_INPUT
    IN          : BOOL;            (* raw input signal from a sensor or switch
*)
    tPresetRise : TIME := T#50MS; (* preset time for rising edge debounce *)
    tPresetFall : TIME := T#50MS; (* preset time for falling edge debounce *)
END_VAR
VAR_OUTPUT
    Q : BOOL; (* debounced "input" signal *)
END_VAR
VAR
    fbDeb : WAS; (* WAS instance for handling the debounce timing *)
END_VAR

```

```
(* Only accept a state change after the raw input remains stable for its
 configured debounce time. *)
fbDeb(IN := IN);
IF fbDeb.ET >= SEL(IN, tPresetFall, tPresetRise) THEN
    Q := IN;
END_IF;
```

1.2.2 POU: DIF

```
FUNCTION_BLOCK DIF
VAR_INPUT
    IN : BOOL;
END_VAR
VAR_OUTPUT
    Q : BOOL;
END_VAR
VAR
    xWas : BOOL;
END_VAR
```

```
Q    := IN <> xWas;
xWas := IN;
```

1.2.3 POU: FirstCycle

```
FUNCTION_BLOCK FirstCycle
VAR_OUTPUT
    Q : BOOL; (* TRUE during the first cycle of the program run *)
END_VAR
VAR
    X : BOOL := FALSE; (* Has executed at least once during the program run
*)
END_VAR
```

```
(* Track the first cycle of the program run. Q will be TRUE only during
 the first cycle, and X will be set to TRUE afterwards. *)
Q := NOT X;
X := TRUE;
```

1.2.4 POU: FLIP_FLOP

```
FUNCTION_BLOCK FLIP_FLOP
VAR_INPUT
    IN : BOOL; (* Input signal *)
END_VAR
VAR_OUTPUT
    Q : BOOL; (* Output signal *)
END_VAR
VAR
    fbTrig : R_TRIG; (* Rising edge trigger function block *)
    xFlip  : BOOL; (* Internal flip-flop state *)
```

```
END_VAR
VAR_TEMP
    xFlop : BOOL; (* Temporary variable for edge detection *)
END_VAR

```

```
fbTrig(CLK := IN, Q => xFlop);

(* Temporarily disable SA0062 warning. The static analyser incorrectly
 flags the R_TRIG function block output as always FALSE. It does not
 have access to the actual run-time behaviour. *)
{analysis -62}
IF xFlop {analysis +62} THEN
    xFlip := NOT xFlip;
END_IF

(* Use a temporary variable for edge detection in order to avoid reading
 the previous state from the output. *)
Q := xFlip;

```

1.2.5 POU: PT1

```
FUNCTION_BLOCK PT1
VAR_INPUT
    IN : REAL;         (* Input signal *)
    P  : REAL := 0.2; (* Proportional factor *)
END_VAR
VAR_OUTPUT
    Q : REAL; (* Output signal *)
END_VAR
VAR
    Q1 : REAL; (* Previous output signal, used for feedback *)
END_VAR
VAR_TEMP
    P1 : REAL; (* Limited proportional factor *)
END_VAR

```

```
(* First, limit the proportional factor to the range [0, 1] because
values outside this range could lead to instability or unexpected
behaviour. Next, calculate the output signal using the limited
proportional factor and the internal state, i.e. the previous output
signal. The current cycle's output becomes the next cycle's feedback. *)
P1 := LIMIT(0, P, 1);
Q1 := (IN * P1) + (Q1 * (1 - P1));
Q  := Q1;

```

1.2.6 POU: SymmetricDebounce

```
FUNCTION_BLOCK SymmetricDebounce
VAR_INPUT
    IN : BOOL;            (* raw input signal from a sensor or switch *)
    PT : TIME := T#50MS;  (* preset time for rising and falling edge debounce
*)
END_VAR
VAR_OUTPUT
    Q : BOOL;  (* debounced "input" signal *)
END_VAR
VAR
    fbAsymmetricDebounce : AsymmetricDebounce;
END_VAR

```

```
fbAsymmetricDebounce(IN := IN, tPresetRise := PT, tPresetFall := PT, Q
=> Q);

```

1.2.7 POU: WAS

```
FUNCTION_BLOCK WAS
VAR_INPUT
    IN : BOOL;  (* input signal *)
END_VAR
VAR_OUTPUT
    Q  : BOOL;  (* "quittierung" for change detection flag *)
    ET : TIME;  (* elapsed time since the last change *)
END_VAR
VAR
    xWas : BOOL;  (* previous input value *)
    tWas : SysTimeRtc.SYSTIME;  (* previous timestamp *)
END_VAR
VAR_TEMP
    xDif : BOOL;  (* difference between the current and previous input value *)
    tNow : SysTimeRtc.SYSTIME;  (* current timestamp *)
END_VAR

```

```
(* Capture the current high-resolution RTC time and determine if the
 input has changed. *)
IF SysTimeRtcHighResGet(tNow) <> CmpErrors.Errors.ERR_OK THEN
    RETURN;
END_IF

xDif := xWas <> IN;

(* Update the previous input and timestamp if a change is detected. *)
```

```
10      IF xDif THEN
11          xWas := IN;
12          tWas := tNow;
13      END_IF
14
15      (* Update output variables. The change detection flag (Q) and the
16      elapsed time since the last change (ET). *)
17      Q  := xDif;
18      ET := ULINT_TO_TIME(tNow - tWas);
19
```

1.2.7.1 Method: FB_Init

```
1       METHOD FB_Init : BOOL
2       VAR_INPUT
3           (* Indicates whether to initialise retained variables *)
4           bInitRetains : BOOL;
5           (* Indicates whether the function block is being initialised as part
6              of a copy-code operation *)
7           bInCopyCode : BOOL;
8       END_VAR
9
```

```
1       (* Get the current high-resolution RTC time. Throw away the real-time
2       system's internal error code. The function block initialiser sets up
3       a meaningful initial timestamp for the elapsed time calculation. *)
4       FB_Init := SysTimeRtcHighResGet(tWas) = CmpErrors.Errors.ERR_OK;
5
```

1.3 Folder: Functions

1.3.1 POU: ISIGNBIT

```
1       FUNCTION ISIGNBIT : BIT
2       VAR_INPUT
3           iValue : INT;
4       END_VAR
5
```

```
1       ISIGNBIT := iValue < INT#0;
2
```

1.3.2 POU: REPSILONEQ

```
FUNCTION REPSILONEQ : BOOL
VAR_INPUT
    rLHS     : REAL;              (* nominal left-hand side *)
    rRHS     : REAL;              (* nominal right-hand side *)
    rEpsilon : REAL := REAL#1e-6; (* allowable difference *)
END_VAR

```

```
(* Error band is inclusive of the epsilon value.
Less than or equal to is close enough, rather than strict equality. *)
REPSILONEQ := ABS(rLHS - rRHS) <= rEpsilon;

```

1.3.3 POU: RSIGNBIT

```
FUNCTION RSIGNBIT : BIT
VAR_INPUT
    rValue : REAL;
END_VAR

```

```
RSIGNBIT := rValue < REAL#0;

```

1.4 Folder: GlobalConstants

1.4.1 Exchange Variable List: Constants

```
{attribute 'qualified_only'}
{attribute 'conditionalshow'}
{attribute 'conditionalshow' := 'DevMode'}
{attribute 'hide'}
VAR_GLOBAL CONSTANT
END_VAR

```

1.5 Folder: GlobalVariables

1.5.1 Exchange Variable List: Variables

```
{attribute 'qualified_only'}
{attribute 'conditionalshow'}
{attribute 'conditionalshow' := 'DevMode'}
{attribute 'hide'}
VAR_GLOBAL
END_VAR

```

1.6 Folder: Interfaces

1.7 Folder: Structs

1.8 Folder: Types

2 Folder: Library Information

2.1 POU: GetLibVersion

```
// This function has been automatically generated from the project information.
FUNCTION GetLibVersion : VERSION
VAR_INPUT

END_VAR

```

```
GetLibVersion.uiMajor := 0;
GetLibVersion.uiMinor := 5;
GetLibVersion.uiServicePack := 12;
GetLibVersion.uiPatch := 6;

```

2.2 POU: GetLibVersionNumber

```
1   // This function has been automatically generated from the project information.
2   FUNCTION GetLibVersionNumber : DWORD
3   VAR_INPUT
4
5   END_VAR
6
```

```
1   GetLibVersionNumber := 330758;
2
```

2.3 POU: IsLibReleased

```
1   // This function has been automatically generated from the project information.
2   FUNCTION IsLibReleased : BOOL
3   VAR_INPUT
4
5   END_VAR
6
```

```
1   IsLibReleased := FALSE;
2
```

3 Library Manager: Library Manager

#Standard
#SysTimeRtc
  #SysTimeCore
    SysTypes2 Interfaces, * (System)
  SysTypes2 Interfaces, * (System)
CmpErrors2 Interfaces, * (System)

4 Project Information: Project Information

File

Name	CannyIEC.library
Location	D:\OneDrive - R2\IEC
Size	939.02 KB (961,552 bytes)
MS-DOS name	D:\OneDrive - R2\IEC\CannyIEC.library
Created	23 September 2026 05:35:32
Changed	12 October 2026 06:40:58
Last access	12 October 2026 06:40:58
Attributes	Archive
Storage format	CODESYS V3.5 SP22 Patch 3

Summary

Company	Roy Ratcliffe
Title	CannyIEC Library
Version	0.5.12.6
Library Categories	
Default namespace	CANNY
Author	Roy Ratcliffe
Description	CannyIEC's function blocks provide input change and timing detection, first-cycle signaling, symmetric and asymmetric debounce, PT1 analog filtering, and edge-triggered toggle behaviour

Properties

Author	[Text] Roy Ratcliffe
Company	[Text] Roy Ratcliffe
DefaultNamespace	[Text] CANNY
Description	[Text] CannyIEC's function blocks provide input change and timing detection, first-cycle signaling, symmetric and asymmetric debounce, PT1 analog filtering, and edge-triggered toggle behaviour
DocFormat	[Text] reStructuredText
IsEndUserLibrary	[Boolean] True
LanguageModelAttribute	[Text] qualified-access-only
Placeholder	[Text] CannyIEC
Project	[Text] CannyIEC
Released	[Boolean] False
Title	[Text] CannyIEC Library
Version	[Version] 0.5.12.6

Statistics

POU	13
DUT	1
Exchange Variable List	2
Library Manager	1
Project Information	1
Folder	10
Method	1

5 : Project Settings

Static Analysis:
Perform static analysis automatically: TRUE

Rules:
Unreachable code (#1): --
Empty objects (#2): --
Empty statements (#3): --
Multiple write access on output (#4): Report as error
Write access from several tasks (#6): Report as error
Address operator on constants (#7): Report as error
Check subrange types (#8): Report as error
Unused return values (#9): Report as error

Arrays with only one component (#10): --
Useless declarations. Struct/Enum with only a single member should not be used (#11): --
Variables which could be declared as constants (#12): --
Declarations with the same variable name (#13): --
Assignment of instances (#14): Report as error
Access to global data via FB_Init (#15): Report as error
Gaps in structures (#16): --
Unusual assignment to pointer variable (#17): Report as error
Unusual bit access (#18): Report as error
Possibly assignment of truncated value to REAL variable (#20): --
Transporting the address of a temporary variable (#21): Report as error
(Possibly) unassigned return value (#22): --
Complex return values (#23): --
Untyped literals / constants (#24): --
Unqualified enumeration constants (#25): Report as error
Possible truncated strings (#26): Report as error
Multiple uses of identifiers (#27): Report as error
Overlapping memory areas (#28): Report as error
Notation in code different to declaration (#29): Report as error
Unused signatures (#31): --
Unused enumeration constants (#32): --
Unused variables (#33): Report as error
Unused input variables (#35): --
Unused output variables (#36): --
Enumerations with incorrect assignment (#34): Report as error
Write access to input variable (#37): Report as error
Read access to output variable (#38): Report as error
Possible division by zero (#40): Report as error
Detect possible loop invariant code (#41): Report as error
Usage of different access paths (#42): Report as error
Use of a global variable in only one POU (#43): Report as error
Declarations with reference to interface (#44): Report as error
Implicit pointer conversions (#19): Report as error
Implicit expanding conversions (#130): Report as error
Explicit narrowing conversions (#133): --
Explicit signed/unsigned conversions (#134): --
Possible use of not initialised interface (#46): Report as error
Invalid addresses and data types (#5): --
Accesses to direct address (#47): Report as error
AT-declarations on direct addresses (#48): Report as error
Comparison operations on BOOL variables (#51): Report as error
Unusual shift operation (#52): Report as error
Too big bitwise shift (#53): Report as error
Comparisons of REAL/LREAL for equality/inequality (#54): --
Unnecessary comparisons of unsigned operands (#55): Report as error
Constant out of valid range (#56): Report as error
Possible loss of decimal places (#57): Report as error
Operations on enumeration variables (#58): Report as error
Comparison operations always returning TRUE or FALSE (#59): Report as error
Zero used as invalid operand (#60): Report as error
Unusual operation on pointer (#61): Report as error
Expression is constant (#62): Report as error
Possibly not 16-bit-compatible operations (#63): Report as error

Addition of pointer (#64): Report as error
Incorrect pointer addition to base size (#65): Report as error
Uses of temporary results (#66): Report as error
Invalid uses of counter variable (#72): Report as error
Uses of inadequate counter variable (#73): --
Upper border is not a constant (#81): Report as error
Missing ELSE (#75): --
Missing enumeration constant (#76): Report as error
Enum type mismatch with CASE expression (#77): Report as error
Missing CASE branches (#78): Report as error
POUs shall have a single point of exit (#90): --
Assignments in conditions (#95): Report as error
Variables greater than 1024 bytes (#100): Report as error
Names with invalid length (#101): --
Access to program/fb variables from the outside (#102): --
Concurrent access on not atomic data (#103): --
Multiple instance calls (#105): --
Virtual method calls in FB_INIT (#106): --
Missing formal parameters (#107): --
Pointer variables (#111): --
Reference variables (#112): --
Variables with data type WSTRING (#113): --
Variables with data type LTIME (#114): --
Variables with data type UNION (#115): --
Variables with data type BIT (#117): --
Object-oriented features (#119): --
Program calls (#120): --
Missing VAR_EXTERNAL declarations (#121): --
Array index defined as expression (#122): --
Usages of INI, ADR or BITADR (#123): --
Unusual shift operation - strict (#147): --
Unusual bit access - strict (#148): --
Statements commented out (#140): Report as error
Possible null pointer dereferences (#39): Report as error
Possible use of not initialised reference (#145): Report as error
Initialisations not using constants (#118): --
Pointer dereferences in declarations (#124): --
References in initializations (#125): --
Violations of lower or upper limits of the metrics (#150): --
Recursive calls (#160): --
Unpacked structure in packed structure (#161): --
Missing comments (#162): --
Nested comments (#163): --
Multiline comments (#164): --
Tasks calling other POUs than programs (#165): --
Max. number of input/output/in-out variables (#166): --
Temporary function block instances (#167): --
Unnecessary assignments (#168): --
Ignored output (#169): Report as error
Address of an output variable should not be taken (#170): Report as error
Enumerations should have the attribute 'strict' (#171): --
Possible attempt to access outside the bounds of the array (#172): Report as error
Suspicious operation on string (#175): --

Cognitive Complexity (#178): Report as error
Coupling between objects (#179): Report as error
The index doesn't cover the whole array (#180): Report as error
Forbidden symbols (#181): Report as error

Naming Conventions:
BOOL (#3): --
BIT (#4): --
BYTE (#5): --
WORD (#6): --
DWORD (#7): --
LWORD (#8): --
SINT (#13): --
INT (#14): --
DINT (#15): --
LINT (#16): --
USINT (#9): --
UINT (#10): --
UDINT (#11): --
ULINT (#12): --
REAL (#17): --
LREAL (#18): --
STRING (#19): --
WSTRING (#20): --
TIME (#21): --
LTIME (#22): --
DATE (#23): --
DATE_AND_TIME (#24): --
TIME_OF_DAY (#25): --
POINTER (#26): --
REFERENCE (#27): --
SUBRANGE (#28): --
ARRAY (#30): --
Function block instance (#31): --
Interface (#36): --
Structure (#32): --
ENUM (#29): --
Alias (#33): --
Variables with data type UNION (#34): --
__XWORD (#35): --
__UXINT (#37): --
__XINT (#38): --
VAR_GLOBAL (#51): --
VAR_GLOBAL CONSTANT (#70): --
VAR_GLOBAL RETAIN (#71): --
VAR_GLOBAL PERSISTENT (#72): --
VAR_GLOBAL RETAIN PERSISTENT (#73): --
Program variables (#53): --
Function block variables (#54): --
Function\Method variables (#55): --
VAR_INPUT (#56): --
VAR_OUTPUT (#57): --
VAR_IN_OUT (#58): --

VAR_STAT (#59): --
VAR_INST (#60): --
VAR_TEMP (#61): --
VAR CONSTANT (#62): --
VAR PERSISTENT (#63): --
VAR RETAIN (#64): --
IO variables (#65): --
PROGRAM (#102): --
FUNCTIONBLOCK (#103): --
FUNCTION (#104): --
METHOD (#105): --
ACTION (#106): --
PROPERTY (#107): --
INTERFACE (#108): --
PRIVATE (#121): --
PROTECTED (#122): --
INTERNAL (#123): --
PUBLIC (#124): --
Structure (#151): --
Enumeration (#152): --
Variables with data type UNION (#153): --
Alias (#154): --
(#161): --

Metrics :
Code size (number of bytes): Off (--, --)
Variables size (number of bytes): On (--, --)
Stack size (number of bytes): On (--, --)
Number of calls: On (--, --)
Called in tasks: Off (--, --)
Used different global Variables: On (--, --)
Number of direct address accesses: On (--, --)
Number of local variables: Off (--, --)
Number of input variables: Off (--, --)
Number of output variables: Off (--, --)
NOS - Number Of Statements: On (--, --)
Percentage of comment: On (--, --)
Complexity (McCabe): On (--, --)
Cognitive Complexity: On (--, --)
DIT - Depth of Inheritance Tree: On (--, --)
NOC - Number Of Children: On (--, --)
RFC - Response For Class: On (--, --)
CBO - Coupling Between Objects: On (--, --)
Complexity of reference (Elshof): Off (--, --)
Lack of Cohesion Of Methods - LCOM: Off (--, --)
Number of SFC branches
: Off (--, --)
Number of SFC steps
: Off (--, --)